VERSION 2.0

PROJECT ATLAS — MASTER FLIGHTLINE CLOCK

Deployment Guide

Appliance-Mode Installation & Configuration Reference

DOCUMENT ID	ATLAS-DEP-001
VERSION	2.0
DATE	3 July 2026
STATUS	Released
AUTHOR	Peter Lochhead Hepburn
REPLACES	ATLAS-DEP-001 v1.0

Covers Mac Mini M4 • Windows PC • Linux Workstation • Appliance-Mode Deployment

Document Control

Field	Value
Document ID	ATLAS-DEP-001
Title	Deployment Guide — Appliance-Mode Installation & Configuration
Project	Project Atlas — Master Flightline Clock
Version	2.0
Date	3 July 2026
Status	Released
Author	Peter Lochhead Hepburn
Reviewed By	Project Atlas Core Team
Classification	Internal — Competition Operations

Revision History

Version	Date	Author	Summary of Changes
1.0	2026-06-01	P. Lochhead Hepburn	Initial release. Basic installation and startup procedures.
1.1	2026-06-20	P. Lochhead Hepburn	Added SQLite persistence notes. Updated directory structure.
2.0	2026-07-03	P. Lochhead Hepburn	Major revision. Appliance-mode operation, auto-boot configuration (macOS/Windows/Linux), HAL backend architecture, Display Type selector, cross-platform notes, hardware migration path, competition-day checklist.

© 2026 Peter Lochhead Hepburn

Table of Contents

1. Introduction & Deployment Overview 4

2. System Requirements 5

3. Installation Procedures 7

3.1 Python Environment Setup 7

3.2 Dependency Installation 8

3.3 Database Initialisation 9

3.4 Directory Structure & Permissions 9

4. Initial Configuration 10

5. Auto-Boot Configuration 12

5.1 macOS: LaunchAgent Setup 12

5.2 Windows: Startup & Task Scheduler 14

5.3 Linux: systemd Service 15

6. Display Type Configuration 16

7. Startup Sequence & Boot Flow 18

8. Operator Interface Guide 19

9. Hardware Migration Path 21

10. Platform-Specific Notes 23

11. Troubleshooting Guide 25

12. Maintenance & Updates

13. Competition Day Checklist

© 2026 Peter Lochhead Hepburn

1. Introduction & Deployment Overview

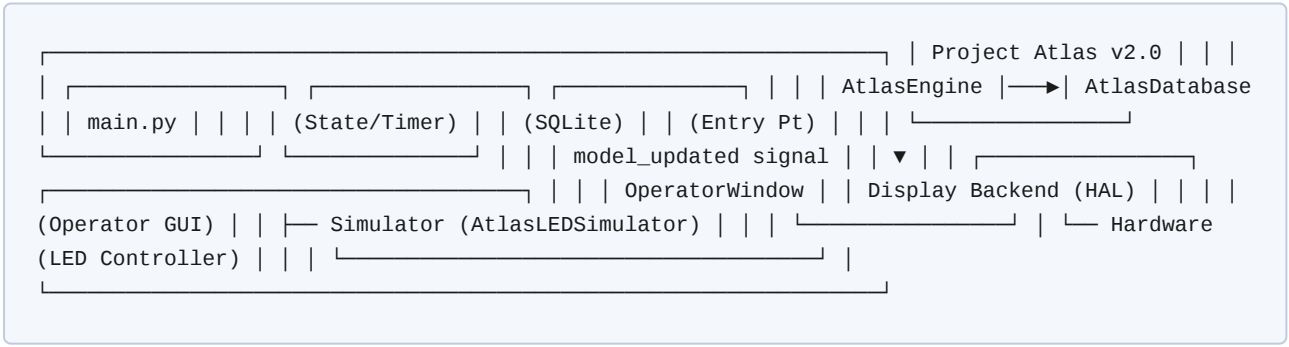
This document is the primary deployment and operations reference for **Project Atlas**, the Master Flightline Clock system for competitive aviation events. Version 2.0 introduces *appliance-mode* deployment as the primary operational model: Atlas starts automatically when the host computer powers on, proceeds directly to clock display mode, and runs without any operator interaction unless configuration changes are required.

This guide supersedes ATLAS-DEP-001 v1.0. It covers all three supported host platforms — Apple Mac Mini, Windows PC, and Linux workstation — from first-time installation through to competition-day operation. Administrators and operators should read Sections 1–4 in full before deployment; Section 13 provides the quick-reference competition-day checklist.

1.1 Appliance-Mode Operation Concept

In appliance mode, the Atlas host computer behaves like a purpose-built timing device rather than a general-purpose desktop. Once configured, the operator does not need a keyboard or mouse to operate the clock during a competition — the system initialises automatically, connects to the database, and begins displaying timing data. Configuration access is reserved for pre-event setup and post-event administration.

The architecture that enables this behaviour is illustrated below. The timing engine, persistence layer, operator interface, and display renderer are separate subsystems that communicate through defined interfaces. This separation means that the display output layer can be replaced or upgraded (for example, swapping the on-screen simulator for physical P5/P10 LED panels) without touching the engine or database.



1.2 Supported Platforms

Atlas is tested and supported on the following host platforms. The **Mac Mini M4** is the primary production target and receives the most thorough hardware integration testing. Windows and Linux are fully supported for development, testing, and alternative production deployments.

Table 1 Supported Deployment Platforms

© 2026 Peter Lochhead Hepburn

Platform	Primary Use	Auto-Boot Method
macOS (Mac Mini M4)	Production appliance	LaunchAgent .plist
Windows 10/11 (PC)	Production / testing	Task Scheduler / Startup folder
Linux (any desktop)	Development / testing	systemd user service

1.3 Deployment Scenarios

Two distinct deployment scenarios exist and govern the appropriate configuration choices:

- **Testing / Development:** Atlas runs on a developer workstation with the LED Simulator (LCD monitor) as the display backend. Auto-boot is optional. The operator uses keyboard and mouse freely and may alter schedules between test runs.
- **Production (Competition):** Atlas runs on a dedicated host — ideally a Mac Mini — with auto-boot enabled and the display backend set to either the LCD Simulator (if physical LED panels are not yet available) or the Hardware backend (P5/P10 LED matrix panels). Operator interaction is kept to a minimum during the event.

2. System Requirements

The following requirements apply to all supported platforms. Where platform-specific requirements differ, they are noted with the relevant badge.

2.1 Hardware Requirements

Table 2 Minimum Hardware Specifications

Component	Minimum	Recommended (Production)
CPU	Dual-core, 1.8 GHz	Apple M4 (Mac Mini M4) or equivalent
RAM	4 GB	8 GB or more
Storage	10 GB free	SSD, 256 GB or more
Network	Not required for core operation	Ethernet for NTP time sync
Display Output	One HDMI / DisplayPort output	Two outputs (operator + public display)
USB	One port (optional, for LED controller)	Two or more USB-A / USB-C ports

© 2026 Peter Lochhead Hepburn

> **NOTE — MAC MINI M4 (PRIMARY PLATFORM)**
>
> The Mac Mini M4 is the preferred production appliance. Its silent fan, compact form factor, and two HDMI outputs make it ideal for both the operator console and a secondary public-facing display. The system has been optimised and tested on this hardware.

2.2 Software Requirements

Table 3 Required Software Components

Component	Version	Notes
Python	3.8 or later (3.12 recommended)	Must include pip and venv modules
PySide6	6.4 or later	Qt6 Python bindings; installed via pip
pydantic	2.x	Data models validation layer
SQLite	3.31 or later	Bundled with Python; no separate install needed
Git (optional)	Any	Recommended for version management

2.3 Display Requirements

Atlas supports two display backends, each with different hardware requirements:

- **LCD Monitor (Simulator backend):** Any standard HDMI or DisplayPort monitor running at 1920×1080 or higher. No additional hardware required. Suitable for development and for competitions where physical LED panels are unavailable.

LED Matrix (Hardware backend — future): P5 or P10 pitch outdoor LED panels driven by a compatible controller (e.g., Colorlight, Nova). Connection via USB or Ethernet depending on controller. Panel array must present a minimum of 64×64 pixels; the standard configuration is 128×64 pixels (two 64×64 P10 panels side by side).

2.4 Operating System Requirements

Table 4 Supported Operating Systems

OS	Versions Tested	Notes
MACOS	Ventura (13), Sonoma (14), Sequoia (15)	Fusion style applied; native tab bars bypassed
WINDOWS	Windows 10 (22H2), Windows 11	Antivirus whitelist recommended (see §10)

© 2026 Peter Lochhead Hepburn

LINUX	Ubuntu 22.04 LTS, Fedora 39+	X11 or Wayland; Qt Fusion style auto-applied

3. Installation Procedures

Follow the steps in this section sequentially on a freshly provisioned host. Estimated total time: 20–30 minutes on a fast internet connection.

3.1 Python Environment Setup

Atlas requires Python 3.8 or later. The recommended approach uses a Python virtual environment (`venv`) to isolate Atlas dependencies from any system-level Python packages.

macOS

1. Open **Terminal** (Applications → Utilities → Terminal).
2. Install Python via Homebrew if not already present:

```
brew install python@3.12
```

3. Navigate to your chosen installation directory and clone or copy the Atlas source tree:

```
cd ~
  mkdir -p Applications/Atlas
  cd Applications/Atlas
  # Copy source files here, or:
  # git clone <repository-url> .
```

4. Create and activate the virtual environment:

```
python3 -m venv .venv
  source .venv/bin/activate
```

Windows

1. Download Python 3.12 from **python.org**. During installation, tick *"Add Python to PATH"* and *"Install for all users"*.
2. Open **Command Prompt** as Administrator and navigate to your chosen directory:

```
cd C:\Atlas
```


© 2026 Peter Lochhead Hepburn

3. Create and activate the virtual environment:

```
python -m venv .venv
  .venv\Scripts\activate
```

Linux

1. Install Python and pip using your distribution's package manager:

```
# Ubuntu/Debian:
  sudo apt update
  sudo apt install -y python3.12 python3.12-venv python3-pip

  # Fedora/RHEL:
  sudo dnf install -y python3.12
```

2. Create and activate the virtual environment:

```
python3.12 -m venv ~/.atlas/.venv
  source ~/.atlas/.venv/bin/activate
```

3.2 Dependency Installation

With the virtual environment active, install all required Python packages. The Atlas project includes a `requirements.txt` file that pins tested dependency versions.

```
# With virtual environment active:
pip install --upgrade pip
pip install -r requirements.txt
```

The `requirements.txt` specifies, at minimum:

```
PySide6>=6.4.0
pydantic>=2.0.0
```

> **WARNING — PYSIDE6 DOWNLOAD SIZE**
>
> PySide6 is a large package (approximately 150–250 MB). Ensure a reliable internet connection before installing. On the Mac Mini M4 with Apple Silicon, ensure you install the `arm64` build; pip will automatically select this when running native Python for Apple Silicon.

 © 2026 Peter Lochhead Hepburn

Verify the installation is complete by running:

```
python -c "import PySide6; import pydantic; print('Dependencies OK')"
```

A successful installation prints: `Dependencies OK`

3.3 Database Initialisation

Atlas uses an embedded SQLite database stored at `data/atlas.db` relative to the application's working directory. The database is created automatically on first launch by the `AtlasDatabase._bootstrap()` method, which creates the `heats` and `settings` tables if they do not already exist.

No manual database creation steps are required. However, the `data/` directory must be writable by the user account running Atlas. To pre-create the directory and verify permissions:

```
mkdir -p data
# macOS/Linux — verify write access:
touch data/.write_test && rm data/.write_test && echo "OK"
```

> **NOTE — DATABASE LOCATION**
>
> The database path is configured in `core/database.py` as `data/atlas.db`. Atlas must always be launched from the project's root directory (the folder containing `main.py`), or the database will be created in the wrong location. The auto-boot scripts in Section 5 handle this by setting a working directory explicitly.

3.4 Directory Structure & Permissions

After installation, the Atlas project directory should contain the following structure:

```
Project_Atlas/ ├── main.py ← Entry point — always launch from this directory ├──
requirements.txt ← Python dependencies ├── run_atlas.sh ← Optional convenience launch script
(macOS/Linux) ├── core/ │ ├── __init__.py │ ├── config.py ← Hardware panel geometry config │
├── database.py ← SQLite persistence layer │ ├── engine.py ← 1 Hz timing state machine │ ├──
models.py ← Pydantic data models │ └── timers.py ← Timer utilities ├── gui/ │ ├── __init__.py │
├── windows.py ← Operator Console (main window) │ ├── themes.py ← Cockpit colour theme
constants │ ├── led_simulator.py ← Virtual LED matrix renderer │ ├── display_backend.py← HAL
interface for future hardware │ └── fonts.py ← Custom 5×7 bitmap font engine ├── data/ │ └──
atlas.db ← SQLite database (auto-created on first run) └── docs/ ← Documentation (SDS, DEP,
URS, Charter)
```


© 2026 Peter Lochhead Hepburn

Ensure the user account running Atlas has **read and write** access to the `data/` directory and **read** access to all Python source files. No files outside the project directory are required or modified during normal operation.

Creating a Convenience Launch Script (macOS / Linux)

Create a script named `run_atlas.sh` in the project root to simplify launching, especially for the auto-boot configuration:

```
#!/bin/bash
# run_atlas.sh — Atlas launch wrapper
cd "$(dirname "$0")"
source .venv/bin/activate
exec python3 main.py "$@"
```

Make it executable:

```
chmod +x run_atlas.sh
```

Test that it launches correctly from Terminal before proceeding to auto-boot configuration.

4. Initial Configuration

After a successful installation, perform the following initial configuration steps before the first competition. These settings are stored in the SQLite database and persist across restarts.

4.1 First Launch

Launch Atlas manually from a terminal for the first configuration session. This ensures any startup errors are visible in the terminal window before auto-boot is configured:

```
# macOS / Linux (from the project root):
source .venv/bin/activate
python3 main.py

# Windows (from the project root):
.venv\Scripts\activate
python main.py
```

On first launch, two windows will appear:

 © 2026 Peter Lochhead Hepburn

- **Atlas Operator Console** — the main control window with three navigation tabs: LIVE, SCHEDULE, and SETTINGS.
- **ATLAS VIRTUAL LED** — the LED simulator window showing the current display output on a virtual matrix panel.

If either window fails to appear, check the terminal for Python error messages and refer to Section 11 (Troubleshooting).

4.2 Competition Name

Navigate to the **SETTINGS** tab by clicking the SETTINGS button in the navigation bar. Enter the competition name in the *Competition Name* field. This value is stored in the settings database and is available for display overlays and future report generation.

4.3 Display Backend Selection

The Display Size setting on the SETTINGS tab controls the virtual LED matrix resolution. Select the profile that matches your intended display hardware:

- **128×64 (Large):** Two 64×64 P10 panels side by side, or the standard full-size simulator window. This is the default and recommended setting for production deployments.
- **64×64 (Compact/Half):** A single 64×64 P10 panel, or any display width of 96 pixels or fewer. The renderer automatically abbreviates label text in compact mode (e.g., "TRACK OPENS IN" becomes "OPEN IN") to maintain readability.

Set the *Sim LED Scale* value to control the on-screen pixel size of the simulator window. A value of 8 produces a comfortable simulator window size on a 1920×1080 monitor. Reduce to 6 if screen space is limited; increase to 10 for higher-resolution displays.

4.4 Brightness Preference

The *Display Intensity* slider controls LED brightness from 10% (Night) to 100% (Full Sun). The slider applies immediately to the simulator and is saved to the database when you click **SAVE & APPLY**. The appropriate brightness levels for different environments are:

Table 5 Brightness Level Guide

Level	Value	Environment
Night	10–20%	Indoor / night operations. Minimum glare.
Overcast	30–40%	Overcast outdoor conditions.

© 2026 Peter Lochhead Hepburn

Normal	50–60%	Standard outdoor daylight.
Bright Day	70–80%	Bright sunny conditions.
Full Sun	90–100%	Maximum output; direct sunlight environments.

4.5 Saving Configuration

Click **SAVE & APPLY** to commit all settings to the database. A confirmation dialog will inform you if a restart is required to apply hardware changes (display size or LED scale). Brightness changes take effect immediately without a restart.

> **NOTE — SETTINGS PERSISTENCE**
>
> All settings are stored in `data/atlas.db` under the `settings` key-value table. There is no separate configuration file. Settings survive application restarts, OS reboots, and software updates, provided the `data/` directory is preserved.

5. Auto-Boot Configuration

Auto-boot places Atlas in true appliance mode: the system starts automatically when the host computer powers on or the user logs in, without any manual terminal or double-click interaction. Configure auto-boot after completing the initial configuration in Section 4 and verifying that Atlas launches and runs correctly from the terminal.

> **PREREQUISITE**
>
> Auto-boot scripts reference the absolute path to the Atlas project directory and the virtual environment. Verify these paths on your specific installation before creating the auto-boot configuration. An incorrect path will silently fail at boot with no visible error.

5.1 macOS — LaunchAgent Setup

MACOS The recommended auto-boot mechanism on macOS is a **LaunchAgent** — a `.plist` configuration file that macOS launches automatically when the user logs in. LaunchAgents run in the user's session with access to the graphical display, making them ideal for Qt applications like Atlas.

Step 1: Enable Automatic Login

For a true appliance with no login prompt, configure macOS Automatic Login:

© 2026 Peter Lochhead Hepburn

1. Open **System Settings → General → Login Items & Extensions** (macOS Ventura+) or **System Preferences → Users & Groups → Login Options** (earlier).
2. Set *Automatic login* to the Atlas operator account.
3. Note: FileVault must be disabled for automatic login to function. If data security is a concern, use FileVault and accept the manual password entry at boot instead.

Step 2: Create the LaunchAgent Plist

Create the following file at `~/Library/LaunchAgents/com.atlas.clock.plist`. Adjust the path to match your actual installation directory.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
    "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>Label</key>
    <string>com.atlas.clock</string>

    <key>ProgramArguments</key>
    <array>
        <string>/Users/atlas/Applications/Atlas/.venv/bin/python3</string>
        <string>/Users/atlas/Applications/Atlas/main.py</string>
    </array>

    <key>WorkingDirectory</key>
    <string>/Users/atlas/Applications/Atlas</string>

    <key>RunAtLoad</key>
    <true/>

    <key>KeepAlive</key>
    <false/>

    <key>StandardOutPath</key>
    <string>/Users/atlas/Applications/Atlas/logs/atlas.log</string>

    <key>StandardErrorPath</key>
    <string>/Users/atlas/Applications/Atlas/logs/atlas_error.log</string>

    <key>EnvironmentVariables</key>
    <dict>
        <key>DISPLAY</key>
        <string>:0</string>
    </dict>
</dict>
</plist>
```

 © 2026 Peter Lochhead Hepburn

Step 3: Create the Logs Directory

```
mkdir -p ~/Applications/Atlas/logs
```

Step 4: Load the LaunchAgent

```
launchctl load ~/Library/LaunchAgents/com.atlas.clock.plist
```

Step 5: Verify

Log out and log back in. Atlas should start automatically. Check the log file if it does not appear:

```
tail -f ~/Applications/Atlas/logs/atlas_error.log
```

Disabling / Enabling Auto-Boot

```
# Disable:
launchctl unload ~/Library/LaunchAgents/com.atlas.clock.plist

# Re-enable:
launchctl load ~/Library/LaunchAgents/com.atlas.clock.plist
```

5.2 Windows — Startup Folder & Task Scheduler

WINDOWS Two options are available on Windows. The **Startup folder** method is simpler; **Task Scheduler** provides more control and is recommended for production deployments.

Option A: Startup Folder (Simple)

1. Create a batch file named `run_atlas.bat` in the project root:

```
@echo off
  cd /d C:\Atlas
  call .venv\Scripts\activate.bat
  python main.py
```

2. Press **Win + R**, type `shell:startup`, and press Enter.
3. Copy a shortcut to `run_atlas.bat` into the Startup folder that opens.
4. Reboot to test.

© 2026 Peter Lochhead Hepburn

Option B: Task Scheduler (Recommended)

1. Open **Task Scheduler** (search the Start menu).
2. Click *Create Basic Task* → name it *Atlas Flightline Clock*.
3. Trigger: *When I log on*.
4. Action: *Start a program*. Set Program to `C:\Atlas\.venv\Scripts\python.exe` and Add Arguments to `main.py`, and Start In to `C:\Atlas`.
5. In *Properties* → *Settings*, untick *"Stop the task if it runs longer than"*.
6. Enable *Run only when user is logged on*.

5.3 Linux — systemd User Service

LINUX On Linux with systemd (Ubuntu, Fedora, and most modern distributions), create a *user service unit* that starts Atlas after the graphical session is available.

Step 1: Create the Service File

Create the file `~/.config/systemd/user/atlas.service`:

```
[Unit]
Description=Atlas Master Flightline Clock
After=graphical-session.target
Wants=graphical-session.target

[Service]
Type=simple
WorkingDirectory=%h/Atlas
ExecStart=%h/Atlas/.venv/bin/python3 %h/Atlas/main.py
Restart=no
Environment=DISPLAY=:0
Environment=DBUS_SESSION_BUS_ADDRESS=autolaunch:
StandardOutput=append:%h/Atlas/logs/atlas.log
StandardError=append:%h/Atlas/logs/atlas_error.log

[Install]
WantedBy=graphical-session.target
```

Step 2: Enable and Start the Service

```
mkdir -p ~/Atlas/logs
systemctl --user daemon-reload
systemctl --user enable atlas.service
systemctl --user start atlas.service
```


© 2026 Peter Lochhead Hepburn

Step 3: Check Status

```
systemctl --user status atlas.service
journalctl --user -u atlas.service -f
```

> **NOTE — WAYLAND COMPATIBILITY**
>
> On systems using Wayland (default in Fedora 38+, Ubuntu 23.10+), set `QT_QPA_PLATFORM=xcb` in the service `Environment` line to force X11 compatibility mode. PySide6 under Wayland may require additional XWayland configuration; X11 mode is the most reliable path for production deployments.

6. Display Type Configuration

Atlas supports two display backends — the software LED Simulator and the physical LED Hardware backend — selectable from the SETTINGS tab. The backend architecture is designed so that both backends receive identical timing data from the engine; only the output rendering layer differs.

6.1 Display Backends Overview

Table 6 Display Backend Comparison

Feature	Simulator (LCD Monitor)	Hardware (LED Matrix)
Status	Available — current production	Pending hardware integration
Hardware required	Any LCD monitor	P5/P10 LED panels + controller
Brightness control	Software dimming of pixel colours	Hardware PWM via controller
Display size	128×64 or 64×64 pixels scaled on screen	Physical panel pixel count
Configuration	Display Size + LED Scale in SETTINGS	Controller IP/COM + panel geometry

6.2 Simulator Backend — LCD Monitor

The simulator renders the LED matrix as a grid of circular "pixels" on a black background within a standard Qt window on the connected LCD monitor. The pixel appearance accurately represents what will be displayed on physical P5/P10 panels — each pixel is drawn as a filled circle with a small gap, matching the physical appearance of LED matrix displays.

© 2026 Peter Lochhead Hepburn

Layout Modes

The simulator automatically selects a layout mode based on the configured display width:

- **Full Mode (128×64 or wider):** Six display rows arranged with the local time at the top, heat information in the centre, and the primary countdown timer in large format at the bottom. The status band (a coloured stripe) indicates the current phase.
- **Compact Mode (64×64 or narrower):** The same six-row layout with abbreviated text labels. The primary timer is displayed at standard scale rather than double scale to accommodate the narrower canvas.

Configuring the Simulator Window Scale

The *Sim LED Scale* setting (in SETTINGS) controls the on-screen pixel size. This does not affect the timing data or display logic — it only changes how large the simulator window appears on screen. Typical values:

- **6–7:** Compact window on 1920×1080; useful when sharing screen space with the Operator Console.
- **8:** Default — comfortable full-size window on 1080p display.
- **10–12:** Large display on high-DPI or 4K monitors. At 128×64 pixels with scale 10, the simulator is approximately 1280×640 px.

6.3 Hardware Backend — Physical LED Panels (Future)

The hardware backend is the planned output path for physical P5 or P10 LED matrix panels driven by a compatible LED controller. The architectural separation of the rendering layer from the engine and database means enabling hardware output requires only replacing or extending the display backend module — no changes to the timing logic are necessary.

The Hardware backend activation process is covered in full in Section 9 (Hardware Migration Path). At the time of this document's publication, the hardware backend module (`gui/display_backend.py`) contains the HAL interface definition and stub implementation. Physical controller communication is a planned future task.

6.4 Testing Display Output Before Competition

Before any competition, verify display output as follows:

1. Launch Atlas and navigate to the SCHEDULE tab.
2. Add a test heat set to open within the next 2–3 minutes.
3. Monitor the LED simulator window and confirm the countdown transitions from WAITING → TRACK OPENS IN → ENTER NOW → HEAT TIME REMAINING → HEAT COMPLETED as each phase threshold is crossed.
4. Adjust brightness using the SETTINGS slider and confirm the simulator updates in real time.

© 2026 Peter Lochhead Hepburn

5. Click the LIVE tab and verify the ABORT / CANCEL button cancels the test heat correctly.
6. Delete the test heat from the SCHEDULE tab before the real competition begins.

7. Startup Sequence & Boot Flow

This section documents the normal startup sequence and the different paths an operator may take depending on whether auto-boot is configured and whether configuration changes are needed before the event.

7.1 Auto-Boot Sequence (Normal Production)

With auto-boot configured, the startup sequence from power-on to active display is as follows:

```
Power On │ ▼ OS boots → User auto-login │ ▼ LaunchAgent / Task Scheduler / systemd fires Atlas
│ ▼ main.py starts ├─ AtlasDatabase() → connects to data/atlas.db ├─ AtlasEngine() → starts 1
Hz QTimer ├─ DisplayConfig() → reads display_width, brightness from DB ├─ OperatorWindow.show()
→ Operator Console appears (LIVE tab) └─ AtlasLEDSimulator.show() → Virtual LED window appears
│ ▼ Engine first tick ├─ No heats scheduled → LED shows "WAITING" └─ Heats scheduled (within 15
min) → auto-activates and begins countdown
```

Total time from power-on to live clock display on a Mac Mini M4: approximately 20–30 seconds (dominated by macOS boot time and Python import time).

7.2 Manual Startup Sequence (Development / Configuration Sessions)

1. Open Terminal.
2. Navigate to the Atlas project root: `cd ~/Applications/Atlas`
3. Activate the virtual environment: `source .venv/bin/activate`
4. Launch: `python3 main.py`
5. Both the Operator Console and LED Simulator windows appear.

7.3 Accessing Settings During Operation

Settings can be accessed at any time during operation without stopping or restarting the clock:

1. Click the **SETTINGS** navigation button in the Operator Console.
2. Make required changes (competition name, brightness, display size).
3. Click **SAVE & APPLY**.
4. If the dialog prompts a restart (for display size/LED scale changes), click **RESTART ATLAS** in the System Controls section. Atlas will restart in approximately 5 seconds and the LED display will resume from the saved state.

© 2026 Peter Lochhead Hepburn

5. For brightness changes — no restart is needed; the simulator updates immediately.

7.4 Safe Shutdown Procedures

Atlas uses a graceful shutdown mechanism that terminates both the Operator Console and LED Simulator windows simultaneously. This prevents "stale" timing data from being visible on the public display after the operator has exited.

Recommended shutdown procedure:

1. Click the **SETTINGS** tab in the Operator Console.
2. Click **EXIT TO macOS** (or *Exit to Desktop* on Windows/Linux).
3. Confirm the shutdown dialog.
4. Both windows close. The LED display goes blank.

> **IMPORTANT — DO NOT FORCE QUIT**
>
> Avoid force-quitting or killing the Atlas process via Activity Monitor or Task Manager. The graceful shutdown ensures the LED display receives a clear signal before the application exits. Force-quitting may leave a static frame on physical LED hardware (when connected), displaying stale timing information to pilots.

8. Operator Interface Guide

The Atlas Operator Console is the main control interface for competition administrators. It uses a custom navigation bar with three tabs: **LIVE**, **SCHEDULE**, and **SETTINGS**. The cockpit-style dark theme and high-contrast colours are optimised for use in outdoor environments with varying light conditions.

8.1 LIVE Tab — Real-Time Monitoring

The LIVE tab displays the current system state and provides emergency controls. It is the default view on startup.

Table 7 LIVE Tab Display States

Display Text	Meaning	LED Colour
WAITING	No heats scheduled, or next heat > 15 min away	Dim grey
H[N] | [DIR] MM:SS	Active heat countdown in progress	Depends on phase

© 2026 Peter Lochhead Hepburn

The **ABORT / CANCEL ACTIVE HEAT** button at the bottom of the LIVE tab immediately cancels the currently active heat. The cancelled heat is flagged as CANCELLED in the database (not deleted), preserving the audit trail. Use this button only in genuine emergencies — cancelled heats cannot be automatically reactivated.

8.2 SCHEDULE Tab — Heat Management

The SCHEDULE tab allows the operator to add, edit, and delete heats for the day's competition. Heats are stored in the SQLite database and survive restarts.

Adding a Heat

1. Enter the heat number in the *Heat Number* field (e.g., `01`, `07A`).
2. Enter the thermalling direction in *Thermalling Dir* (e.g., `LEFT` or `RIGHT`).
3. Set the Track Opens, Track Closes, and Heat Ends times using the time pickers. Times are for today's date.
4. Click **ADD NEW HEAT**.

Editing a Heat

Click any row in the schedule table to load that heat's values into the form. The button changes to **UPDATE SELECTED HEAT**. Modify values and click the button to save.

Deleting a Heat

Select a heat in the table, then click the **DELETE** button that appears. Confirm the deletion dialog. Deleted heats are permanently removed from the database.

> **WARNING — DELETING VS. CANCELLING**
>
> **Delete** removes the heat permanently with no audit trail. **Abort/Cancel** (from the LIVE tab) flags the heat as CANCELLED, preserving it in the database for post-event review. Use Cancel during a competition; use Delete only when correcting a data entry error.

8.3 SETTINGS Tab — Configuration Controls

The SETTINGS tab provides access to all persistent configuration options and system controls.

System Controls

At the bottom of the SETTINGS tab, two system control buttons are available:

© 2026 Peter Lochhead Hepburn

- **RESTART ATLAS:** Performs a hot restart using `os.execv()`, replacing the running process with a fresh Atlas instance. Settings changes that require a restart (display size, LED scale) take effect after this action. The database is not affected.
- **EXIT TO macOS / Exit to Desktop:** Initiates graceful shutdown, closing all Atlas windows cleanly. On macOS the button is labelled *EXIT TO macOS*; on Windows it reads *Exit to Desktop*; on Linux it reads *Exit*. This label is determined at runtime based on the operating system.

8.4 Emergency Shutdown Procedures

If the Operator Console becomes unresponsive and the EXIT button is inaccessible, use the following emergency procedures:

- MACOS Right-click the Python icon in the Dock → Force Quit, or use Activity Monitor to quit the `python3` process.
- WINDOWS Press Ctrl+Alt+Delete → Task Manager → find `python.exe` → End Task.
- LINUX Open a terminal and run: `pkill -f main.py`

9. Hardware Migration Path

This section describes the path from the current simulator-based deployment to a physical LED matrix display system. The Atlas architecture was deliberately designed to make this transition require only the rendering layer — the timing engine, database, and operator interface remain unchanged.

9.1 Architecture of the Hardware Abstraction Layer (HAL)

The display backend is separated from the application logic through a Hardware Abstraction Layer defined in `gui/display_backend.py`. The HAL interface specifies a standard contract that any display backend must implement:

- **update_model(model: AtlasDisplayModel)** — called on every engine tick with the current display state.
- **set_brightness(value: int)** — live brightness update (0–100).
- **show()** — initialise and display the output.
- **close()** — graceful shutdown; send blank/clear frame to hardware.

The `AtlasLEDSimulator` class currently implements this interface for the LCD monitor backend. A future `AtlasLEDHardware` class will implement the same interface for physical LED controllers, allowing the two backends to be swapped by changing a single line in `main.py`.

© 2026 Peter Lochhead Hepburn

9.2 Transitioning from Simulator to Physical LED Panels

Phase 1: Hardware Acquisition and Bench Testing

1. Procure P5 or P10 pitch LED matrix panels in the required configuration (recommended: two 64×64 P10 panels = 128×64 pixel total).
2. Procure a compatible LED controller (Colorlight 5A-75B, Linsn RV908M32, or equivalent).
3. Connect panels to the controller and power supply per the controller manufacturer's instructions.
4. Verify the panel array is functional using the controller's native test software before connecting to Atlas.

Phase 2: Backend Module Development

1. Implement `AtlasLEDHardware` in `gui/display_backend.py`, conforming to the HAL interface.
2. The hardware module must translate the `AtlasDisplayModel` into the pixel frame format expected by the LED controller protocol (typically a flat RGB byte array).
3. Use the existing bitmap font in `gui/fonts.py` (AtlasBitmapFont.FONT_5x7) for text rendering — the same font used by the simulator, ensuring visual consistency between simulator and hardware output.
4. Implement the `close()` method to send an all-black frame to the panels on shutdown, preventing stale display.

Phase 3: Integration into main.py

To activate the hardware backend, modify `main.py` as follows:

```
# Replace this line:
from gui.led_simulator import AtlasLEDSimulator

# With:
from gui.display_backend import AtlasLEDHardware as AtlasLEDSimulator
```

No other changes to `main.py` are required, as the hardware backend implements the same interface as the simulator.

9.3 Hardware Backend Activation Checklist

- ☐ LED panels powered on and displaying controller test pattern
- ☐ Controller connected to Atlas host via USB or Ethernet
- ☐ Controller driver installed (if USB connection)

© 2026 Peter Lochhead Hepburn

- ☐ AtlasLEDHardware module implemented and unit-tested against AtlasDisplayModel
- ☐ main.py import line updated to use hardware backend
- ☐ Atlas launched; "WAITING" text visible on physical panels
- ☐ Test heat entered; all five phase transitions confirmed on physical display
- ☐ Brightness control verified via SETTINGS slider
- ☐ Graceful shutdown tested; panels go dark on Atlas exit

9.4 Rollback Procedures if Hardware Fails

If physical LED hardware fails during a competition, revert to the simulator backend immediately:

1. Exit Atlas (SETTINGS → EXIT TO macOS).
2. Open a terminal and edit `main.py`: restore the `AtlasLEDSimulator` import.
3. Relaunch Atlas: `python3 main.py`
4. The LED simulator window will appear on the operator's LCD monitor, which can serve as a fallback display visible to nearby observers.
5. Log the hardware failure in the event record for post-event diagnosis.

> **DESIGN PRINCIPLE**
>
> The HAL separation means a hardware failure never affects the timing engine or database. Heat schedules, completed heat records, and configuration are all preserved regardless of which display backend is active.

10. Platform-Specific Notes

10.1 macOS — Security, Gatekeeper & Permissions

MACOS Modern macOS applies several security controls that can affect Atlas on a fresh deployment.

Gatekeeper — "Unidentified Developer" Warning

If Atlas is distributed as a folder rather than a signed application bundle, macOS Gatekeeper will block execution the first time a script is run. To resolve:

1. Right-click (or Control-click) the script file → Open.
2. Click *Open* in the Gatekeeper dialog.

© 2026 Peter Lochhead Hepburn

3. Alternatively, remove the quarantine attribute: `xattr -rd com.apple.quarantine ~/Applications/Atlas`

Accessibility Permissions

PySide6 Qt applications may request Accessibility permissions on macOS Ventura and later when using certain widget interactions. Grant these permissions in **System Settings → Privacy & Security → Accessibility** if prompted.

Screen Recording Permission (Dual-Monitor Setup)

If the LED simulator is displayed on a secondary monitor and a screen recording or streaming application is also running, macOS may request Screen Recording permission for Python. Grant this in **System Settings → Privacy & Security → Screen & System Audio Recording**.

NTP Time Synchronisation

Atlas computes phase transitions by comparing the current system time against programmed heat timestamps. The system time must be accurate. On macOS, ensure Network Time is enabled: **System Settings → General → Date & Time → Set time automatically**. The Mac Mini M4 should be connected to the venue network or internet for NTP synchronisation before the competition begins.

10.2 Windows — Firewall and Antivirus Configuration

WINDOWS Windows Defender and third-party antivirus software may interfere with Atlas in several ways.

Windows Defender SmartScreen

When first launching `run_atlas.bat` or a Python script, Windows SmartScreen may display a blue warning dialog. Click *More info → Run anyway* to proceed. This warning appears only once per script.

Antivirus Exclusions

Some antivirus products flag Python virtual environments as suspicious due to the large number of DLL files in `.venv\Lib\`. If Atlas launches slowly or fails to start, add the Atlas project folder to the antivirus exclusion list:

1. Open Windows Security → Virus & Threat Protection.
2. Under *Virus & threat protection settings*, select *Manage settings*.
3. Scroll to *Exclusions → Add or remove exclusions*.
4. Add the folder `C:\Atlas` (or wherever Atlas is installed).

© 2026 Peter Lochhead Hepburn

10.3 Linux — Display Server Compatibility

LINUX Atlas uses PySide6 (Qt6) which supports both X11 and Wayland display servers. However, for production deployments the X11 (XCB) backend is strongly recommended for stability and predictable window placement.

Forcing X11 Mode on Wayland Systems

Set the following environment variable before launching Atlas, or include it in the systemd service file:

```
export QT_QPA_PLATFORM=xcb
python3 main.py
```

Missing Qt Platform Plugin

If Atlas fails with the error *"Could not load the Qt platform plugin 'xcb'"*, install the missing X11 libraries:

```
# Ubuntu/Debian:
sudo apt install -y libxcb-cursor0 libxcb-xinerama0 libxkbcommon-x11-0

# Fedora:
sudo dnf install -y xcb-util-cursor
```

11. Troubleshooting Guide

This section provides diagnostic and resolution steps for the most common issues encountered during deployment and operation. For issues not listed here, consult the log file at `logs/atlas.log` for error messages.

Table 8 Troubleshooting Quick Reference

Symptom	Probable Cause	Section
Auto-boot does not start Atlas	Incorrect path in plist/task/service	11.1
Atlas starts but LED window missing	Display backend initialisation error	11.2
Countdown not updating	System time desynchronised or heat timestamps wrong	11.3
Database error on startup	Missing data/ directory or permission denied	11.4
Settings not saving	data/atlas.db not writable	11.4

© 2026 Peter Lochhead Hepburn

Qt platform plugin error	Missing X11 libraries (Linux)	11.5

11.1 Auto-Boot Not Working

macOS LaunchAgent Failures

The most common cause is an incorrect path in the .plist file. Diagnose with:

```
# Check if the agent is loaded:
launchctl list | grep atlas

# View error output:
cat ~/Applications/Atlas/logs/atlas_error.log

# Manually trigger the agent:
launchctl start com.atlas.clock
```

If the agent is listed but Atlas does not appear, the most likely cause is that the working directory or Python path in the plist is incorrect. Verify by running the exact Python path from a terminal:

```
/Users/atlas/Applications/Atlas/.venv/bin/python3 --version
```

If this fails, the virtual environment path is wrong. Update the plist and reload.

Windows Task Scheduler Failures

Open Task Scheduler, find the Atlas task, and check the *Last Run Result*. Common result codes:

- `0x1` — Script returned error; check `run_atlas.bat` paths.
- `0x41303` — Task has not been run yet; trigger manually with *Run*.
- `0xC000013A` — Application was terminated; check for antivirus interference.

Linux systemd Failures

```
systemctl --user status atlas.service
journalctl --user -u atlas.service --since "1 hour ago"
```

The most common cause is the service starting before the display server (X11/Wayland) is ready. Add a delay:

```
ExecStartPre=/bin/sleep 5
```


© 2026 Peter Lochhead Hepburn

11.2 Display Backend Issues

If the Operator Console appears but the LED simulator window does not:

1. Check the terminal for import errors from `gui/led_simulator.py`.
2. Verify PySide6 is installed: `python3 -c "from PySide6.QtWidgets import QApplication"`
3. On macOS with multiple monitors: check that Atlas is not opening the LED window on a disconnected display. Unplug and replug external monitors and restart Atlas.

11.3 Database Corruption Recovery

If Atlas fails to start with a database error such as *"database disk image is malformed"*:

1. Stop Atlas immediately.
2. Create a backup of the corrupted database:

```
cp data/atlas.db data/atlas.db.corrupted.$(date +%Y%m%d%H%M%S)
```

3. Attempt SQLite recovery:

```
sqlite3 data/atlas.db ".recover" | sqlite3 data/atlas_recovered.db
  mv data/atlas.db data/atlas.db.bak
  mv data/atlas_recovered.db data/atlas.db
```

4. If recovery fails, delete `data/atlas.db` and restart Atlas. A fresh database will be created. Re-enter the competition schedule from backup records.

> **COMPETITION RISK**
>
> Database corruption is unlikely but most commonly caused by a sudden power loss during a write operation. Use a UPS (Uninterruptible Power Supply) for the Atlas host during competitions to eliminate this risk. Always maintain a manual paper backup of the heat schedule.

11.4 Permission Denied Errors

If Atlas cannot write to the database or log files:

© 2026 Peter Lochhead Hepburn

```
# macOS / Linux — fix data directory permissions:
chmod 755 ~/Applications/Atlas/data
chmod 644 ~/Applications/Atlas/data/atlas.db

# Check the owner:
ls -la ~/Applications/Atlas/data/
```

Ensure the running user account owns the `data/` directory. If Atlas is running as a different user (e.g., via a system-level LaunchDaemon rather than a LaunchAgent), the file ownership will not match.

11.5 Platform-Specific Errors

Table 9 Platform-Specific Error Messages

Platform	Error Message	Resolution
macOS	"This application cannot be opened" (Gatekeeper)	Right-click → Open, or use xattr -rd (see §10.1)
Windows	"python.exe is not recognized"	Python not in PATH; reinstall with "Add to PATH" ticked
Linux	"xcb plugin not found"	Install libxcb packages (see §10.3)
All	"ModuleNotFoundError: PySide6"	Virtual environment not activated; run source .venv/bin/activate
All	"OperationalError: no such table: heats"	Wrong working directory; ensure you launch from the project root

12. Maintenance & Updates

12.1 Software Update Procedures

Atlas software updates are distributed as revised source code. The update process preserves the existing database and all stored heat records and settings.

Standard Update Procedure

1. Exit Atlas (SETTINGS → EXIT TO macOS / Exit to Desktop).

© 2026 Peter Lochhead Hepburn

2. Back up the current installation directory (especially `data/atlas.db`):

```
cp -r ~/Applications/Atlas ~/Applications/Atlas.backup.$(date +%Y%m%d)
```

3. If using Git, pull the latest changes:

```
cd ~/Applications/Atlas
  git pull origin main
```

If distributing as a folder copy, replace source files only — do **not** overwrite the `data/` directory.

4. Update Python dependencies:

```
source .venv/bin/activate
  pip install -r requirements.txt --upgrade
```

5. Relaunch Atlas and verify normal operation.
6. Check the SCHEDULE tab to confirm heat data is intact.

> **DATABASE SCHEMA CHANGES**
>
> If an update includes database schema changes, the `AtlasDatabase._bootstrap()` method will apply them on first launch using `CREATE TABLE IF NOT EXISTS` statements. Existing data is preserved. If a breaking schema change is introduced in a future release, migration notes will be included in the release documentation.

12.2 Database Backup and Restore

Manual Backup

```
# Create a timestamped backup:
cp data/atlas.db data/atlas.db.$(date +%Y%m%d_%H%M%S).bak
```

Automated Daily Backup (macOS cron)

Add the following to the atlas user's crontab (`crontab -e`) to create a nightly backup:

```
# Backup Atlas database nightly at 02:00
0 2 * * * cp ~/Applications/Atlas/data/atlas.db \
    ~/Applications/Atlas/data/backups/atlas.db.$(date +\%Y\%m\%d).bak
```


© 2026 Peter Lochhead Hepburn

Restoring from Backup

1. Exit Atlas.
2. Copy the backup file to `data/atlas.db`:

```
cp data/atlas.db.20260703_120000.bak data/atlas.db
```

3. Relaunch Atlas and verify the schedule is restored.

12.3 Log File Locations and Analysis

Atlas writes stdout and stderr to log files when launched via auto-boot scripts. In terminal sessions, output appears directly in the terminal.

Table 10 Log File Locations

Platform	Log File	Contains
macOS (LaunchAgent)	`~/Applications/Atlas/logs/atlas.log`	Startup messages, engine ticks
macOS (LaunchAgent)	`~/Applications/Atlas/logs/atlas_error.log`	Python exceptions, import errors
Linux (systemd)	`journalctl --user -u atlas.service`	All output, systemd events
Windows (Task Scheduler)	Task History tab in Task Scheduler	Last run result code

12.4 Performance Optimisation

Atlas is a lightweight application with a 1 Hz update cycle. Performance issues are rare, but the following tips apply if sluggish behaviour is observed:

- **Close unnecessary applications:** On the dedicated Mac Mini, disable Spotlight indexing of the Atlas folder (`System Settings → Siri & Spotlight → Spotlight Privacy`) and close all non-Atlas applications.
- **Reduce LED simulator scale:** Higher LED scale values increase the number of pixels rendered per frame. Reduce the Sim LED Scale value in SETTINGS to improve rendering performance on low-powered hardware.

© 2026 Peter Lochhead Hepburn

- **Database maintenance:** After many events, the `heats` table may accumulate thousands of historical records. Periodically export and archive old records using the SQLite command-line tool:

```
sqlite3 data/atlas.db "DELETE FROM heats WHERE status IN ('COMPLETED','CANCELLED')
  AND heat_end < strftime('%s','now','-30 days');"
```

13. Competition Day Checklist

Use this checklist on competition day to verify Atlas is configured, connected, and operating correctly before the first heat. Each item should be confirmed by the system operator and initialled on the printed copy.

13.1 T-60 Minutes: Pre-Event Preparation

☐Atlas host computer powered on and booted to desktop

☐Atlas application launched (manually or via auto-boot)

☐Operator Console LIVE tab visible on operator monitor

☐LED Simulator (or physical LED display) showing "WAITING"

☐System clock verified against reference time source (phone, GPS, official timekeeper)

☐NTP synchronisation confirmed active (macOS: System Settings → Date & Time)

☐Database backup created: `cp data/atlas.db data/atlas.db.pre-event.bak`

☐Logs directory exists and is writable

13.2 T-30 Minutes: Schedule Entry

☐Navigate to SCHEDULE tab in Operator Console

☐Enter all heats for today's competition with correct times

☐Verify Track Open → Track Close → Heat End times are in correct sequence

☐Confirm heat count matches official competition briefing sheet

☐Add a test heat 5 minutes from now — verify countdown appears correctly — then delete it

☐Cross-check thermalling directions for each heat against briefing

© 2026 Peter Lochhead Hepburn

13.3 T-15 Minutes: Display Verification

- ☐ LED simulator brightness set to appropriate level for ambient conditions
- ☐ Physical LED panels (if connected) displaying correctly and with readable text
- ☐ LED display visible from the competition viewing area (conduct field check)
- ☐ Backup display plan confirmed (what happens if primary display fails)
- ☐ Paper backup of heat schedule printed and available at the control point

13.4 During Competition: Operational Procedures

- ☐ Monitor LIVE tab at all times during active heats
- ☐ If a heat must be cancelled, use ABORT / CANCEL ACTIVE HEAT — do not delete
- ☐ After each heat completes, verify status shows COMPLETED in SCHEDULE table
- ☐ If brightness adjustment is needed: SETTINGS → adjust slider → SAVE & APPLY (no restart required)
- ☐ Do not restart Atlas during an active heat

13.5 Post-Event: Shutdown and Archival

- ☐ After final heat marked COMPLETED, click SETTINGS → EXIT TO macOS (or Desktop)
- ☐ Confirm both Operator Console and LED windows have closed
- ☐ Create post-event database archive:

```
cp data/atlas.db data/atlas.db.$(date +%Y%m%d).post-event.bak
```

- ☐ Power off LED panels before powering off the Atlas host
- ☐ Document any anomalies or incidents in the event log
- ☐ Submit post-event database copy to competition records

> **END OF DOCUMENT**
>
> ATLAS-DEP-001 v2.0 — Deployment Guide. For technical issues beyond this document's scope, refer to ATLAS-SDS-001 (Software Design Specification) for architectural detail, or contact the Product Owner: Peter

© 2026 Peter Lochhead Hepburn

> Lochhead Hepburn.

© 2026 Peter Lochhead Hepburn